

ROBOTICS & ARTIFICIAL INTELLIGENCE DEPARTMENT

Total Marks: _____

Obtained Marks: _____

Computing Vision
Lab Task # 12

Submitted To: Mr. Muhammad Azhar

Student Name: Ayesha Munir

Reg. Number: 23108106

Code Example 1: Auto Scaling in Medical Image Feature Analysis

A team of radiologists is working with a dataset of extracted features from brain MRI images to classify the presence of tumors. Two features are considered:

- Feature1: Tumor area in mm^2
- Feature2: Average pixel intensity in the suspected region

Problem: The raw feature values have different units and scales — tumor area ranges from 10 to 50, while pixel intensities range from 160 to 200. Feeding such data directly into a machine learning model may cause bias towards larger-scale features, degrading performance.

Solution: The team uses StandardScaler (Z-score normalization) to scale features to a common distribution with mean 0 and standard deviation 1. This improves the accuracy of SVM and logistic regression classifiers used for tumor classification.

```
# Code Example 1: Auto Scaling in Medical Image Feature Analysis
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
```

```
data = pd.DataFrame({
    'Feature1': [10, 20, 30, 40, 50],
    'Feature2': [200, 190, 180, 170, 160]
})

scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)

scaled_df = pd.DataFrame(scaled_data, columns=data.columns)

print("Original Data:\n", data)
print("\nScaled Data:\n", scaled_df)

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.title("Original Features")
plt.plot(data['Feature1'], label='Feature1')
plt.plot(data['Feature2'], label='Feature2')
plt.legend()

plt.subplot(1, 2, 2)
plt.title("Scaled Features")
plt.plot(scaled_df['Feature1'], label='Feature1 (scaled)')
plt.plot(scaled_df['Feature2'], label='Feature2 (scaled)')
plt.legend()

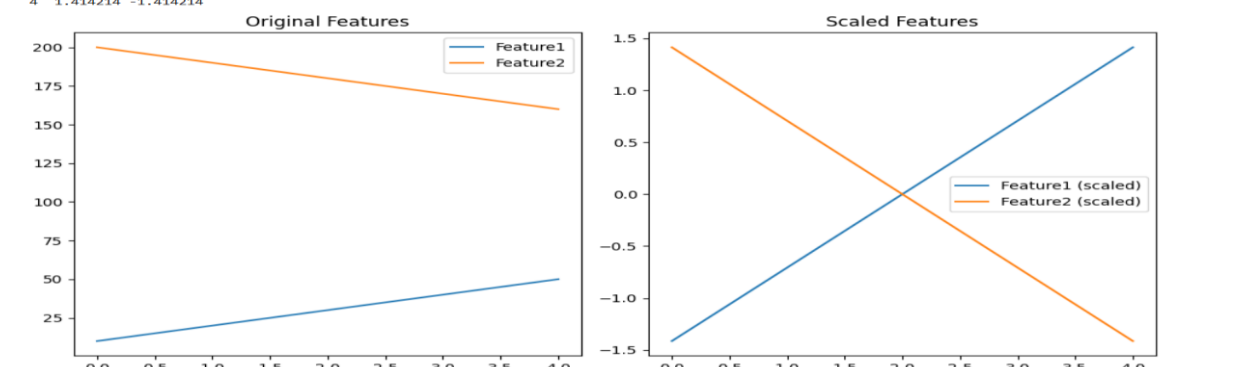
plt.tight_layout()
plt.show()
```

Original Data:

	Feature1	Feature2
0	10	200
1	20	190
2	30	180
3	40	170
4	50	160

Scaled Data:

	Feature1	Feature2
0	-1.414214	1.414214
1	-0.707107	0.707107
2	0.000000	0.000000
3	0.707107	-0.707107
4	1.414214	-1.414214



Code Example 2: Detecting Cyclic Patterns in Industrial Sensor Data

A manufacturing plant uses a vibration sensor to monitor the operation of a rotating machine. Every second, the sensor records the vibration level of the machine. Engineers suspect that there might be a repeating pattern (cyclic behavior) in the machine's operation that could indicate early signs of wear or mechanical imbalance.

Objective

Use autocorrelation analysis to identify if the sensor data exhibits a repetitive pattern over time, which could hint at a periodic mechanical issue.

```
: # Code Example 2: Detecting Cyclic Patterns in Industrial Sensor Data
import matplotlib.pyplot as plt
import pandas as pd

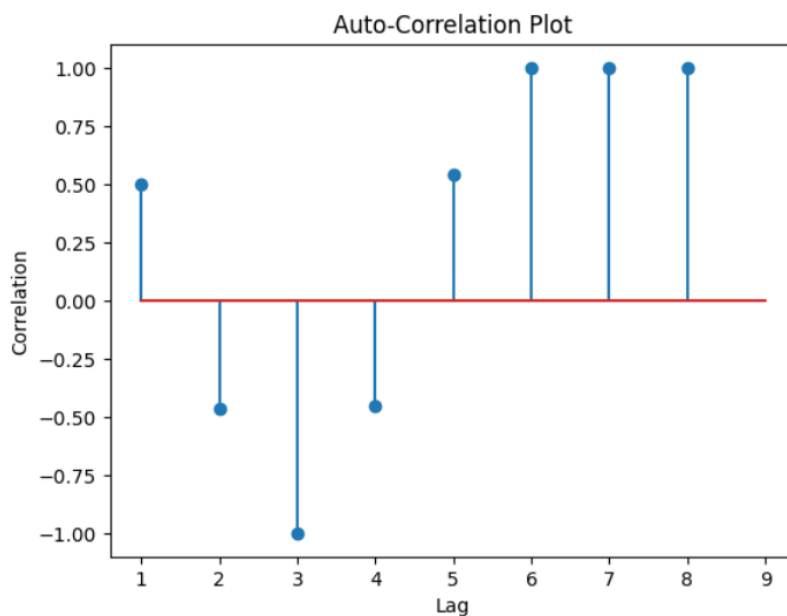
data = pd.Series([1, 2, 3, 4, 3, 2, 1, 2, 3, 4])

autocorrelation = [data.autocorr(lag=i) for i in range(1, len(data))]

plt.stem(range(1, len(data)), autocorrelation)

plt.title("Auto-Correlation Plot")
plt.xlabel("Lag")
plt.ylabel("Correlation")

plt.show()
```



Code Example 3: Identifying Repetitive Patterns in a Production Line Sensor

Company: AutoLine Robotics Pvt. Ltd. **Department:** Production Engineering **Machine:** Robotic arm used in assembly line operations **Sensor:** Vibration sensor attached to the base of the robotic arm **Frequency of data collection:** Every second

Business Problem

Recently, maintenance engineers noticed a pattern of unusual wear and tear in a specific robotic arm. While there was no immediate fault detected, the wear appeared in cyclical bursts. They hypothesized that the arm may be undergoing hidden repetitive stress not visible in standard inspection. To validate this, they decided to analyze vibration data using autocorrelation to detect any underlying patterns.

```
# Code Example 3: Identifying Repetitive Patterns in a Production Line Sensor
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)

base_pattern = [1, 2, 3, 4, 3, 2, 1, 2, 3, 4]
noise = np.random.normal(0, 0.2, len(base_pattern))

vibration_data = np.array(base_pattern) + noise

data = pd.Series(vibration_data)

autocorrelation = [data.autocorr(lag=i) for i in range(1, len(data))]

plt.figure(figsize=(10, 5))
plt.plot(data, markers='o', label='Vibration Data')

plt.title("Robotic Arm Vibration Signal (10 Seconds)")
plt.xlabel("Time (seconds)")
plt.ylabel("Vibration Level")

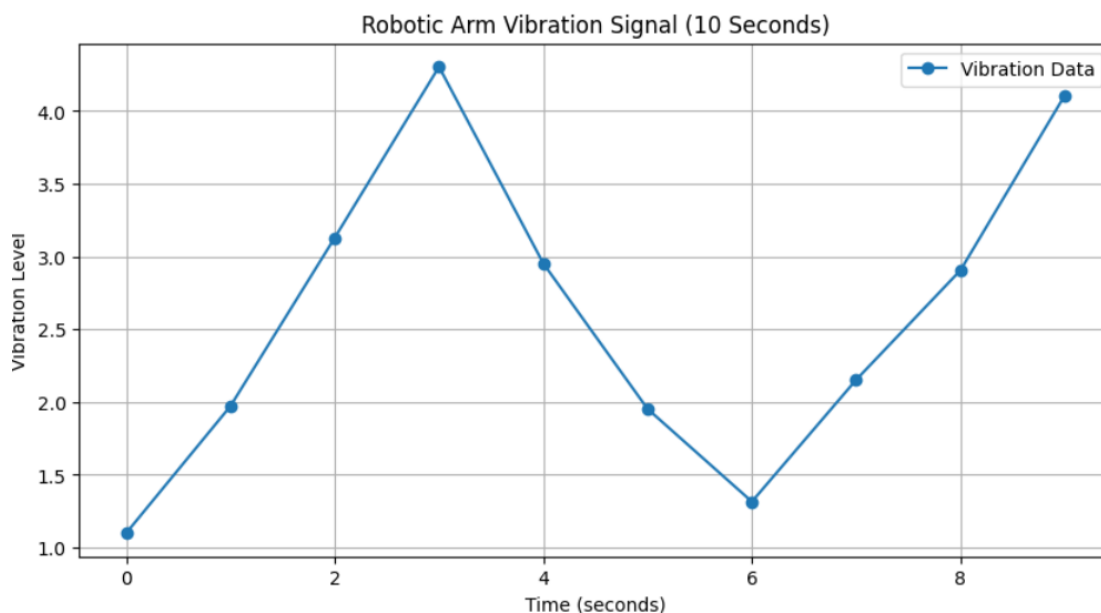
plt.grid(True)
plt.legend()

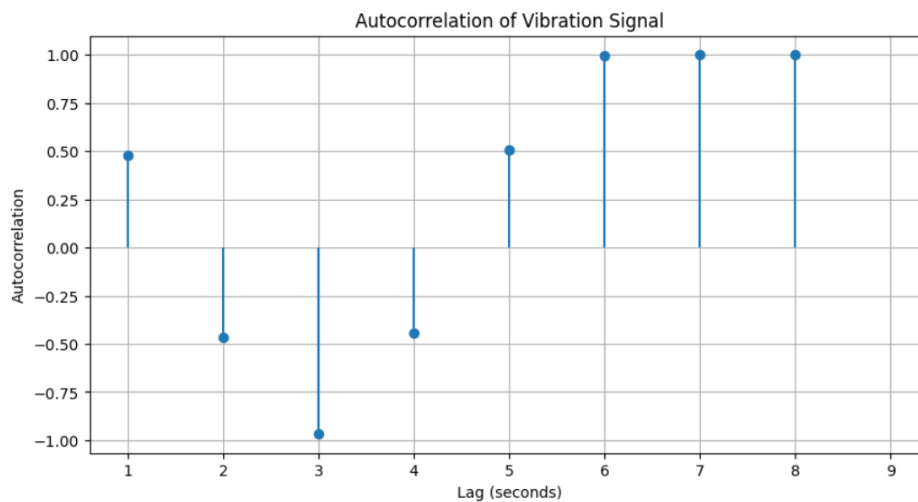
plt.show()

plt.figure(figsize=(10, 5))
plt.stem(
    range(1, len(data)),
    autocorrelation,
    basefmt=" "
)

plt.title("Autocorrelation of Vibration Signal")
plt.xlabel("Lag (seconds)")
plt.ylabel("Autocorrelation")

plt.grid(True)
```





Case Study 1: Medical Diagnosis Data Normalization

A hospital is building a machine learning model to classify patient health conditions based on vital signs such as:

- ☐ Heart rate (in bpm),
- ☐ Body temperature (in °C),
- ☐ Blood pressure (in mmHg).

However, these values have different scales, which leads to poor model performance. To fix this, they apply data normalization (auto-scaling) using StandardScaler.

```
#Case Study 1: Medical Diagnosis Data Normalization
import pandas as pd
from sklearn.preprocessing import StandardScaler

data = {
    'HeartRate_bpm': [72, 88, 60, 102, 77],
    'Temperature_C': [36.6, 37.8, 36.1, 38.2, 37.0],
    'BloodPressure_mmHg': [120, 135, 110, 145, 125]
}

df = pd.DataFrame(data)

scaler = StandardScaler()
normalized_data = scaler.fit_transform(df)

normalized_df = pd.DataFrame(normalized_data, columns=df.columns)

print("Normalized Patient Data:")
print(normalized_df)
```

```
Normalized Patient Data:
   HeartRate_bpm  Temperature_C  BloodPressure_mmHg
0      -0.545895      -0.702782      -0.579324
1       0.573890       0.858956       0.662085
2      -1.385734      -1.353506      -1.406930
3       1.553702       1.379535       1.489691
4      -0.195962      -0.182203      -0.165521
```

Case Study 2: Seasonal Pattern Detection in Weather Data

A meteorological department wants to analyze daily temperature patterns to detect seasonal trends and cycles (e.g., summers and winters). By using autocorrelation, they aim to identify the seasonal lag.

```
# Case Study 2: Seasonal Pattern Detection in Weather Data
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)

days = 30

temperature = (
    20
    + 5 * np.sin(2 * np.pi * np.arange(days) / 7)
    + np.random.normal(0, 1, days)
)

temp_series = pd.Series(temperature)

autocorr_values = [
    temp_series.autocorr(lag=i)
    for i in range(1, 16)
]

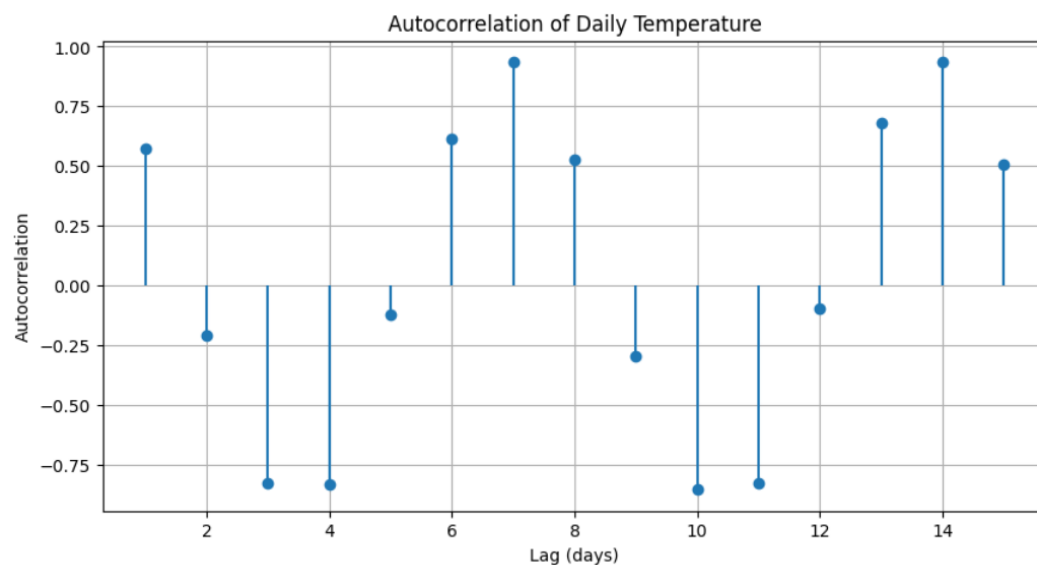
plt.figure(figsize=(10, 5))

plt.stem(
    range(1, 16),
    autocorr_values,
    basefmt=" "
)

plt.title("Autocorrelation of Daily Temperature")
plt.xlabel("Lag (days)")
plt.ylabel("Autocorrelation")

plt.grid(True)

plt.show()
```



Case Study 3: Credit Scoring in Finance

A bank uses features like credit history, income, and age to determine a person's credit score. However, not all features are equally important:

- ❑ Credit history is more predictive than age.
- ❑ Feature weighting improves model accuracy in logistic regression.

```
# Case Study 3: Credit Scoring in Finance
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

data = {
    'CreditHistoryScore': [650, 700, 620, 800, 720],
    'Income_k': [30, 45, 25, 70, 50],
    'Age': [25, 35, 22, 45, 33],
    'LoanApproved': [0, 1, 0, 1, 1]
}

df = pd.DataFrame(data)

X = df[['CreditHistoryScore', 'Income_k', 'Age']]
y = df['LoanApproved']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = LogisticRegression()
model.fit(X_scaled, y)

coefficients = pd.Series(
    model.coef_[0],
    index=X.columns
)

print("Feature Weights (Logistic Regression):")
print(coefficients.sort_values(ascending=False))
```

```
Feature Weights (Logistic Regression):
Age                0.649174
Income_k           0.622297
CreditHistoryScore 0.588569
dtype: float64
```

Case Study 1: Patient Health Risk Prediction (Healthcare)

Technique: Auto Scaling (Normalization)

Scenario

A hospital uses machine learning to predict patient risks for cardiac events based on features like blood pressure, cholesterol, heart rate, and glucose level. These variables have different units and scales.

Solution

They applied auto scaling (e.g., StandardScaler) to normalize all input features before feeding them into a logistic regression model. This ensured that no feature dominated due to its magnitude.

Outcome

The prediction model became more balanced and accurate, helping doctors prioritize high-risk patients.

```
# Case Study 1: Patient Health Risk Prediction (Healthcare)
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

data = {
    'BloodPressure': [120, 140, 130, 150, 110],
    'Cholesterol': [200, 240, 180, 260, 170],
    'HeartRate': [72, 88, 65, 95, 70],
    'Glucose': [90, 110, 85, 130, 80],
    'Risk': [0, 1, 0, 1, 0]
}

df = pd.DataFrame(data)

X = df[['BloodPressure', 'Cholesterol', 'HeartRate', 'Glucose']]
y = df['Risk']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = LogisticRegression()
model.fit(X_scaled, y)

print("Model Coefficients:")
print(model.coef_)

Model Coefficients:
[[0.48299071 0.53880621 0.58508542 0.49591896]]
```

Case Study 2: Seasonal Crop Yield Forecasting (Agriculture)

Technique: Auto Correlation

Scenario

An agritech company monitors monthly crop yields and weather conditions like rainfall and temperature. They suspect seasonal cycles affect crop production.

Solution

They used auto-correlation to analyze historical yield data and weather patterns. The analysis revealed strong yearly cycles, especially linked to monsoon timing.

Outcome

The insights helped farmers plan sowing and fertilization more effectively, increasing yield by ~15%.

```
# Case Study 2: Seasonal Crop Yield Forecasting (Agriculture)
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)

months = 36

yield_data = (
    50
    + 10 * np.sin(2 * np.pi * np.arange(months) / 12)
    + np.random.normal(0, 2, months)
)

crop_series = pd.Series(yield_data)

autocorr_values = [
    crop_series.autocorr(lag=i)
    for i in range(1, 13)
]

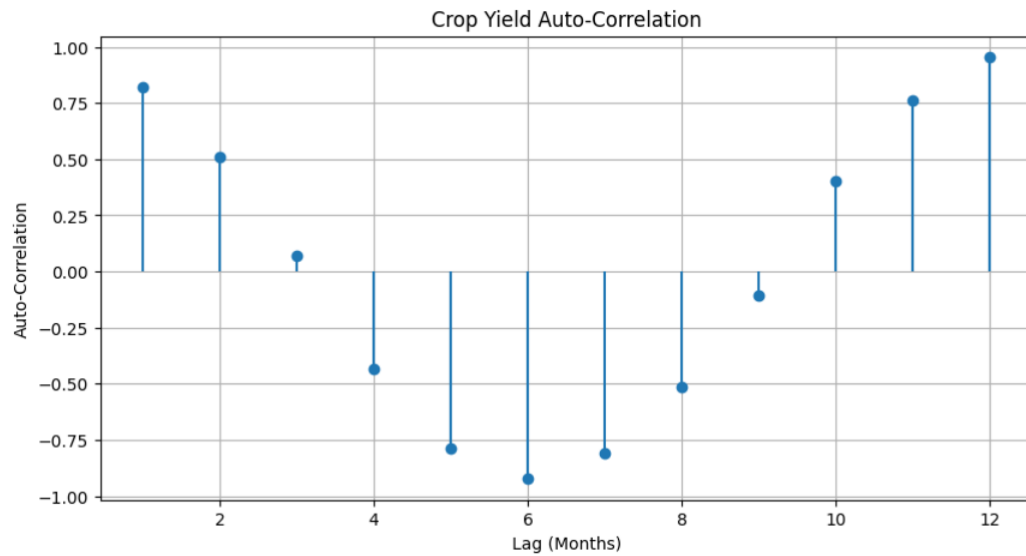
plt.figure(figsize=(10, 5))

plt.stem(
    range(1, 13),
    autocorr_values,
    basefmt=" "
)

plt.title("Crop Yield Auto-Correlation")
plt.xlabel("Lag (Months)")
plt.ylabel("Auto-Correlation")

plt.grid(True)

plt.show()
```

Case Study 3: Creditworthiness Assessment (Finance)

Technique: Feature Weighting

Scenario

A financial institution wants to automate loan approval. Inputs include income, credit history, employment length, and city of residence.

Solution

They used feature weighting via logistic regression coefficients to assign higher importance to credit history and income and lower to demographic variables.

Outcome

Model precision improved, and the bank reduced loan default rates by 12%.

```
# Case Study 3: Creditworthiness Assessment (Finance)
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

data = {
    'Income': [30, 50, 25, 70, 45],
    'CreditHistory': [650, 720, 600, 800, 690],
    'EmploymentLength': [2, 5, 1, 10, 4],
    'LoanApproved': [0, 1, 0, 1, 1]
}

df = pd.DataFrame(data)

X = df[['Income', 'CreditHistory', 'EmploymentLength']]
y = df['LoanApproved']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = LogisticRegression()
model.fit(X_scaled, y)

weights = pd.Series(
    model.coef_[0],
    index=X.columns
)

print("Feature Weights:")
print(weights.sort_values(ascending=False))
```

Feature	Weight
Income	0.677734
CreditHistory	0.598297
EmploymentLength	0.520522
dtype:	float64

Case Study 4: Anomaly Detection in Industrial Machines (Manufacturing)

Techniques: Auto Correlation + Auto Scaling

Scenario

An automotive plant wants to detect irregular machine vibrations in robotic arms that indicate impending faults.

Solution

1. Auto-scaling was applied to sensor data (vibration, temperature, torque).
2. Auto-correlation was used to find repeating anomalies in time-series data.

Outcome

The company shifted to predictive maintenance, reducing machine downtime by 30%.

```
# Case Study 4: Anomaly Detection in Industrial Machines (Manufacturing)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

np.random.seed(42)

vibration = [1, 2, 3, 4, 3, 2, 1, 2, 3, 7]
temperature = [40, 41, 42, 43, 42, 41, 40, 41, 42, 50]
torque = [10, 12, 14, 15, 14, 12, 10, 11, 13, 20]

df = pd.DataFrame({
    'Vibration': vibration,
    'Temperature': temperature,
    'Torque': torque
})

scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)

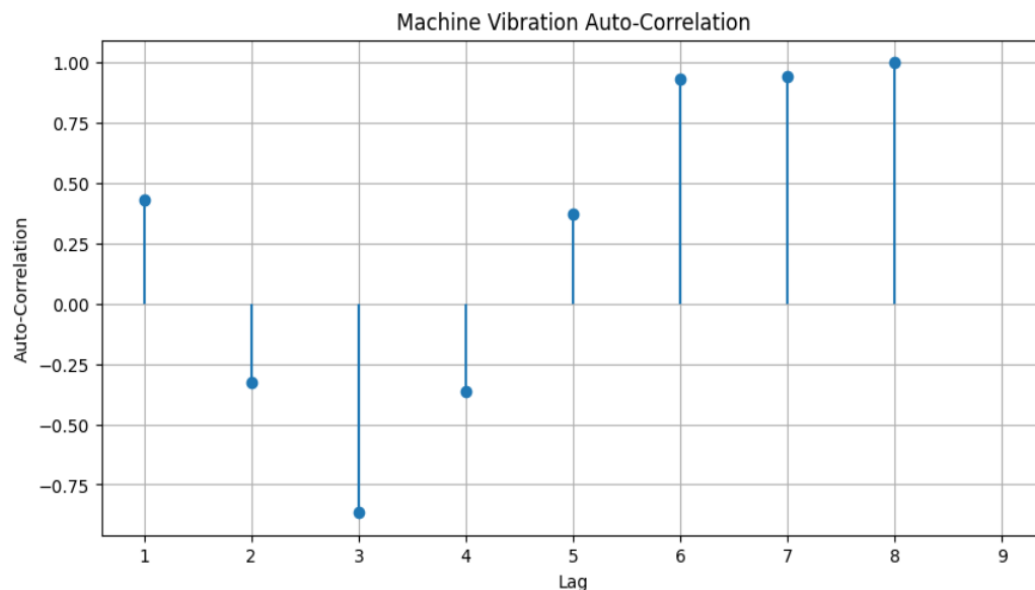
scaled_df = pd.DataFrame(
    scaled_data,
    columns=df.columns
)

autocorr_values = [
    scaled_df['Vibration'].autocorr(lag=i)
    for i in range(1, len(df))
]

plt.figure(figsize=(10, 5))

plt.stem(
    range(1, len(df)),
    autocorr_values,
    basefmt=" "
)

plt.title("Machine Vibration Auto-Correlation")
```



Case Study 5: Intrusion Detection in Network Security (Cybersecurity)

Techniques: Feature Weighting + Auto Scaling

Scenario

A cybersecurity firm analyzes millions of log records to detect network intrusions based on attributes like packet size, duration, and login success rate.

Solution

- Applied auto-scaling to ensure fair comparison between numeric features.
- Used feature selection/weighting (e.g., Random Forest importance) to identify and boost influential variables like unusual port access.

Outcome

The detection rate of abnormal network activity increased, and false positives dropped significantly.

```
: # Case Study 5: Intrusion Detection in Network Security (Cybersecurity)
```

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
```

```
data = {
    'PacketSize': [500, 1500, 300, 2000, 100],
    'Duration': [2, 10, 1, 15, 1],
    'LoginSuccessRate': [0.9, 0.2, 0.95, 0.1, 0.98],
    'UnusualPortAccess': [0, 1, 0, 1, 0],
    'Intrusion': [0, 1, 0, 1, 0]
}
```

```
df = pd.DataFrame(data)
```

```
X = df.drop('Intrusion', axis=1)
y = df['Intrusion']
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
```

```
model.fit(X_scaled, y)
```

```
importance = pd.Series(
    model.feature_importances_,
    index=X.columns
)
```

```
print("Feature Importance:")
print(importance.sort_values(ascending=False))
```

```
Feature Importance:
Duration      0.296703
PacketSize   0.252747
LoginSuccessRate 0.252747
UnusualPortAccess 0.197802
dtype: float64
```

<https://github.com/Ayesha-Munir-Chohan/>